CSE 4137

Machine Learning
Module-07
Deep Learning



Md. Shahid Uz Zaman
Adjunct Faculty
Dept of CSE, BAUET
szaman22.ruet@gmail.com

Topics: KNN

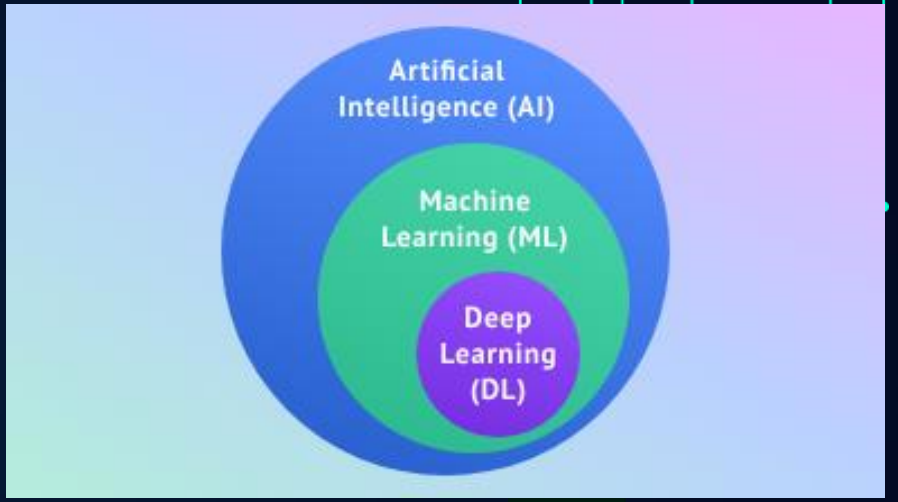
Week-13

- Introduction
- Implementation
- Python Example

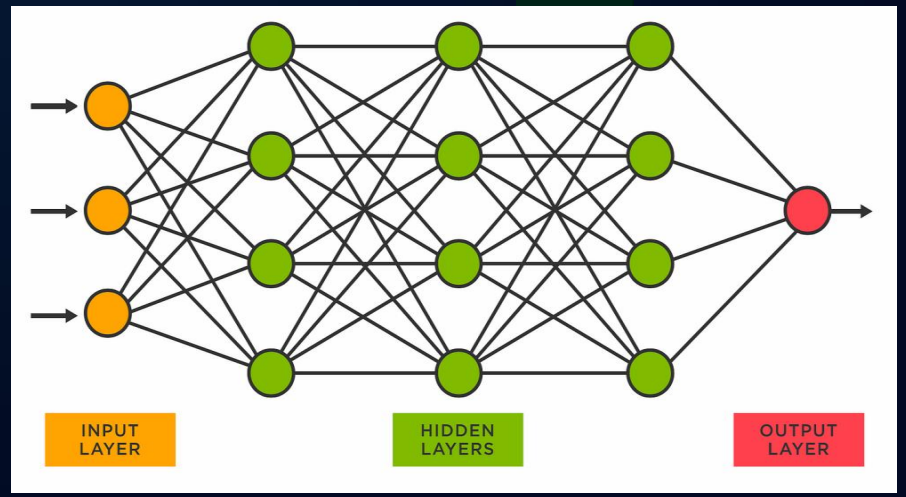
Introduction

What is Deep Learning?

- Deep Learning is a subfield of Artificial Intelligence and Machine Learning that is inspired by the structure of human brain.
- Deep Learning algorithm attempt to draw similar conclusions as human would try continually analyzing data with a given logical structure called Neural Network.



- Machine Learning: Uses statistical technique
- Deep Learning: Uses logical structure inspired by human brain

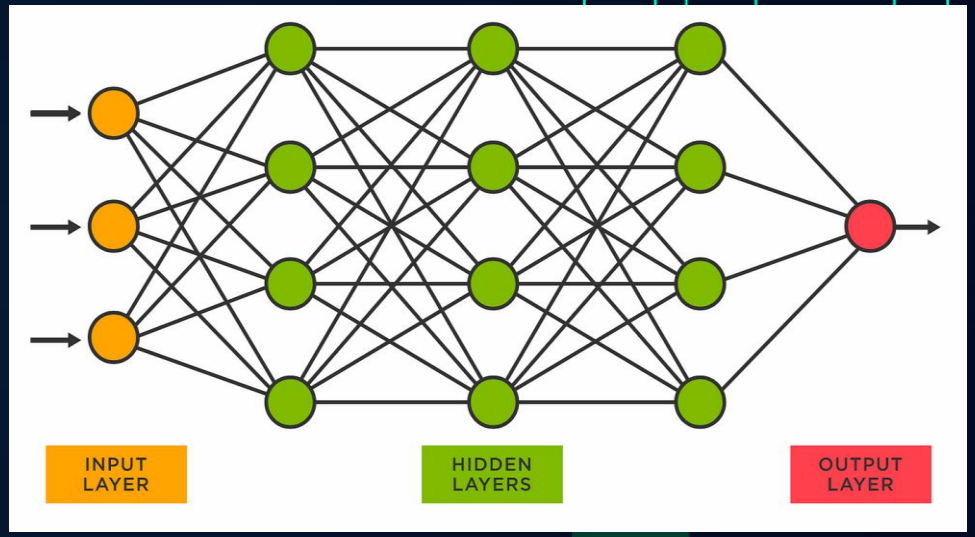


Technical Definition of Deep Learning

- Deep Learning is part of a broader family of ML methods based on ANN with Representative Learning

- Deep Learning algorithms uses multiple layers to progressively extract higher level features from the raw input.

  - Machine Learning: Features are given
  - Deep Learning: Features are automatically extracted

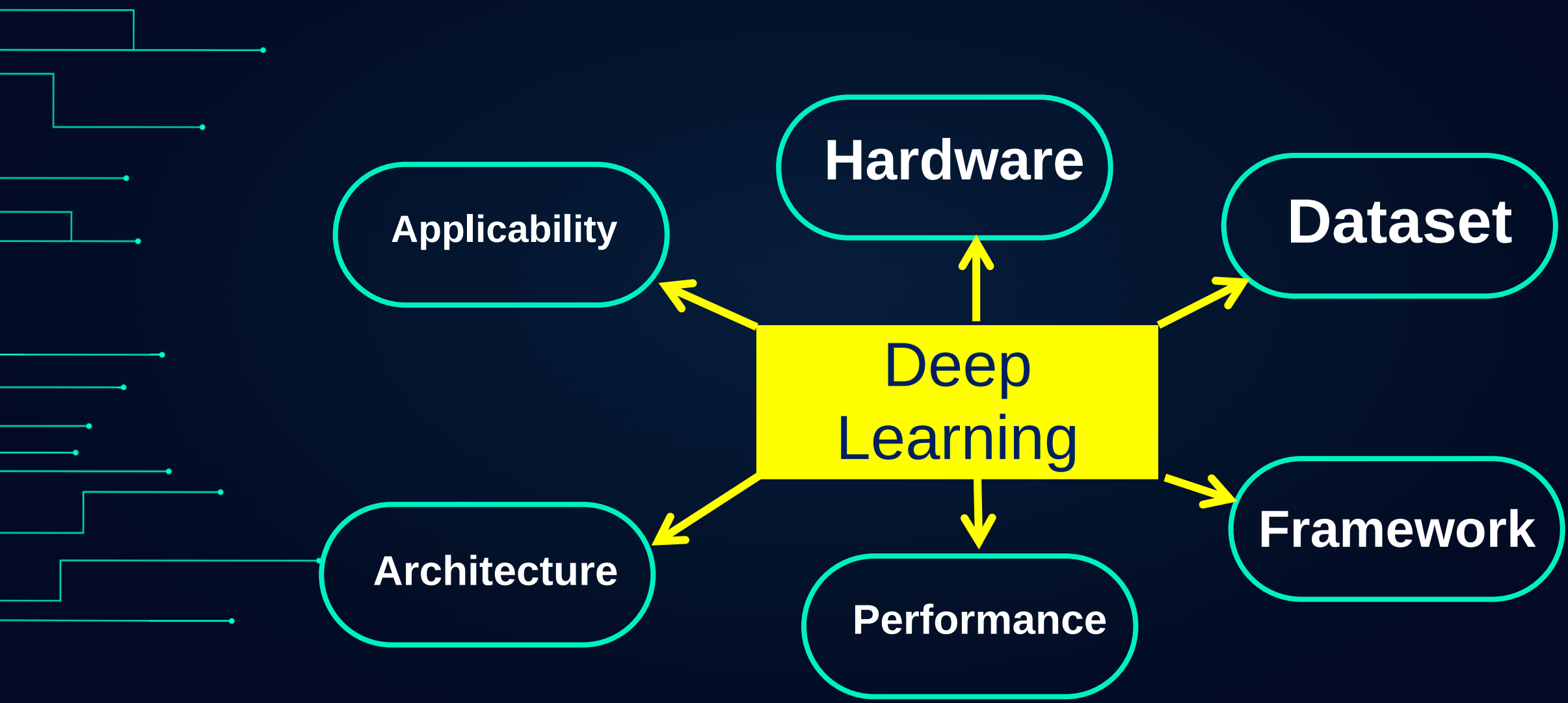


Machine Learning vs Deep Learning

	Machine Learning	Deep Leaning
Data Dependency	Low	High
Hardware Requirement	Low	High
Training Time	Low	High
Feature Selection	Manual	Automatic
Interpretability	More	Less

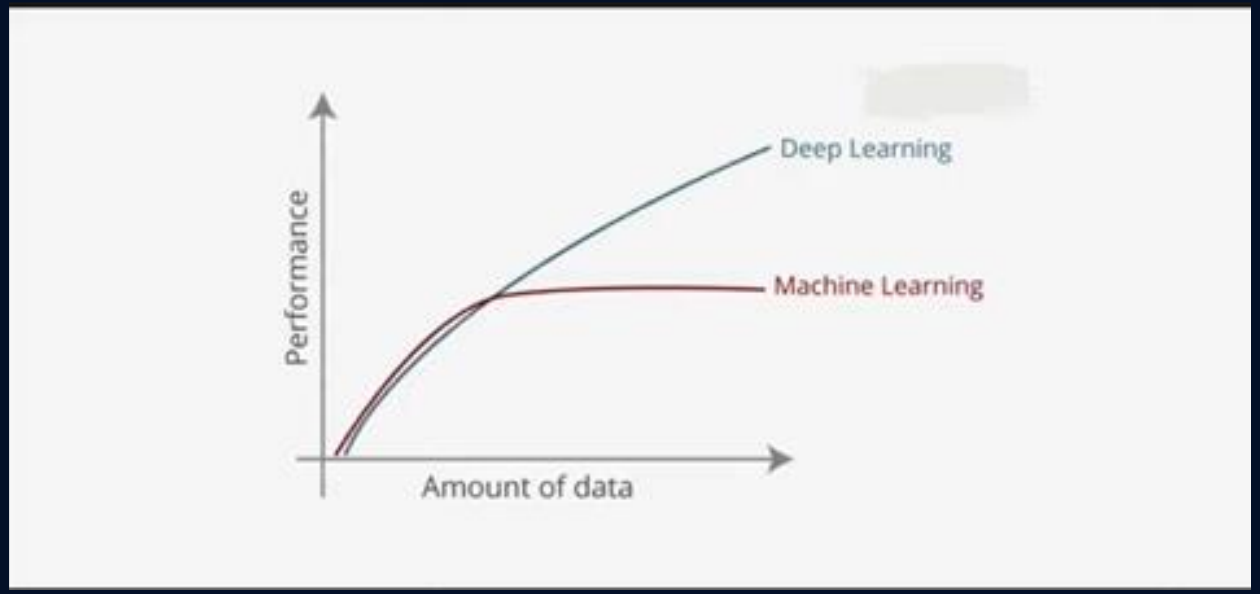
Why DL Getting Famous?

Why Deep Learning Getting Famous?



Datasets

- Data hungry
- Smart Phone & Internet Pricing Revolution after 2010
- We generates huge data daily basis
- Google, Facebook, Microsoft etc create labels of data

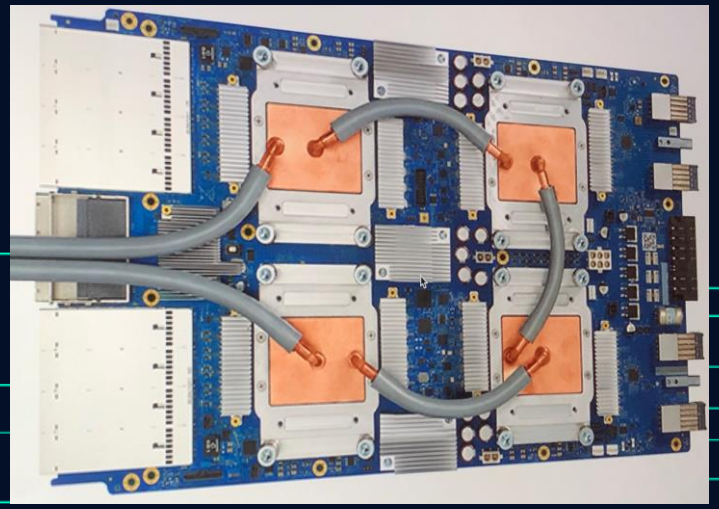


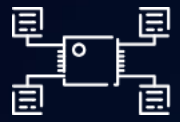
Hardware

- DL requires high performance hardware
- GPU with CPU used in DL
- FPGA-fast, low powered and configurable (MS Bing)
- TPU (Tensor Processing Unit) by Google
- NPU (Neural Processing Unit) for mobile device









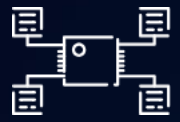
Frameworks

- DL Library: Tensorflow (Google) & Pytorch(Facebook)
- Keras library works top on Tensorflow
- AI researcher uses Pytorch more
- Industry developer uses Tensorflow more



Architecture

- Different architecture can be made based on problem
- Existing architecture can be used (Transfer Learning)
- State-of-art architecture like ResNet, BERT, Unet, Pix2Pix, Yolo)



Community

- People, teachers, students, researchers, engineers move DL forward



Applicability

- Applicable in very wide range of area



Performance

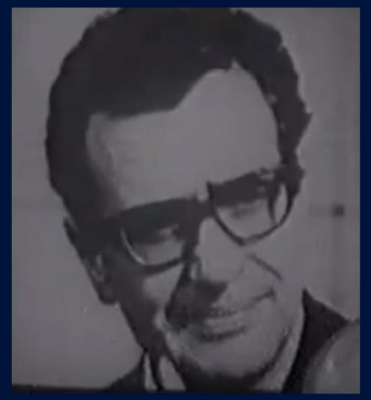
- State-of-art results are found

History

History of Deep Learning

- Frank Rosenblatt first developed Neural Network that separates male and female from photos in 1957







NEW NAVY DEVICE LEARNS BY DOING

Psychologist Shows Embryo of Computer Designed to Read and Grow Wiser

WASHINGTON, July 7 (UPI) —The Navy revealed the embryo of an electronic computer today that it expects will be able to walk, talk, see, write, reproduce itself and be conscious of its existence.

The embryo—the Weather Bureau's $2,000,000 "704" computer—learned to differentiate between right and left after fifty attempts in the Navy's ings, Perceptron will make mistakes at first, but will grow wiser as it gains experience, he said.

Dr. Rosenblatt, a research psychologist at the Cornell Aeronautical Laboratory, Buffalo, said Perceptrons might be fired to the planets as mechanical space explorers.

Without Human Controls

The Navy said the perceptron would be the first non-living mechanism "capable of receiving, recognizing and identifying its surroundings without any human training or control."

The "brain" is designed to remember images and information it has perceived itself. Ordinary computers remember only what is fed into them on punch

1958 New York Times...

In today's demonstration, the "704" was fed two cards, one with squares marked on the left side and the other with squares on the right side.

Learns by Doing

In the first fifty trials, the machine made no distinction between them. It then started registering a "Q" for the left squares and "O" for the right squares.

Dr. Rosenblatt said he could explain why the machine learned only in highly technical

History of Deep Learning

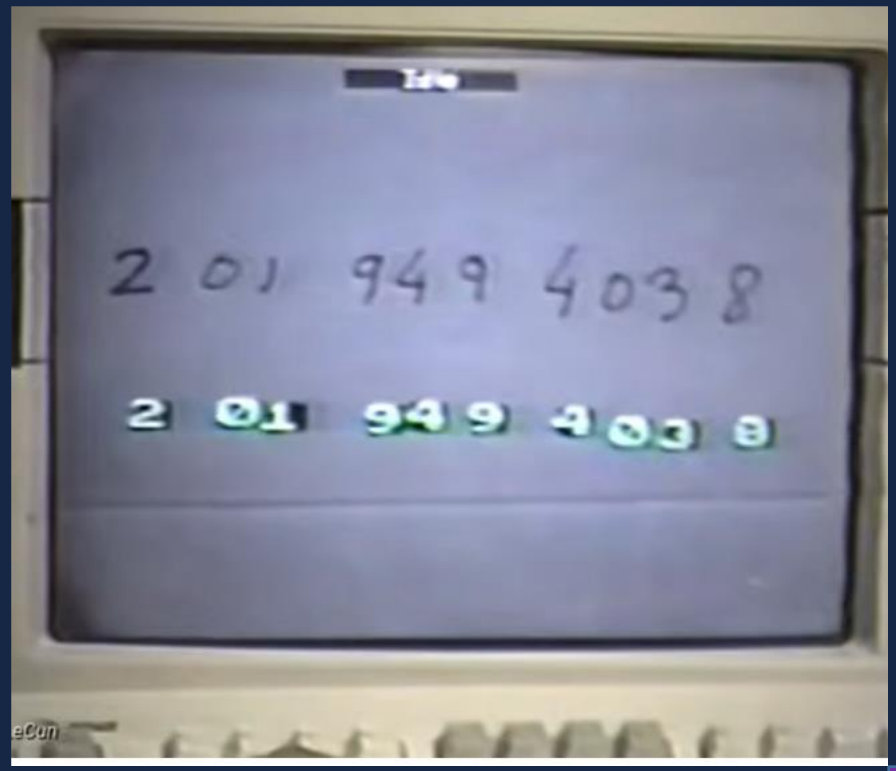
1986

Self-driven car
Speed: 2Km/hour



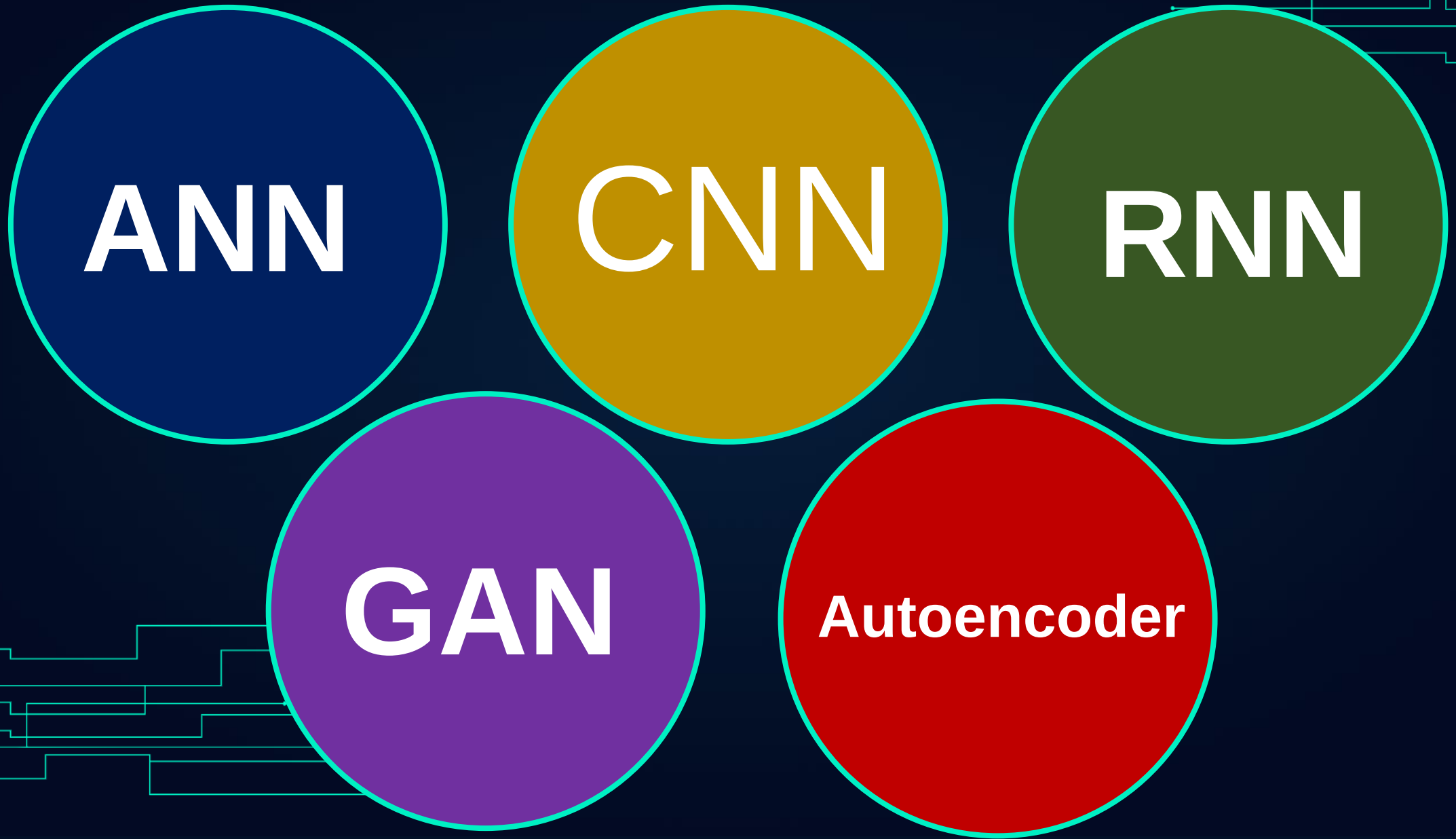
1984

Recognize Hand-written characters



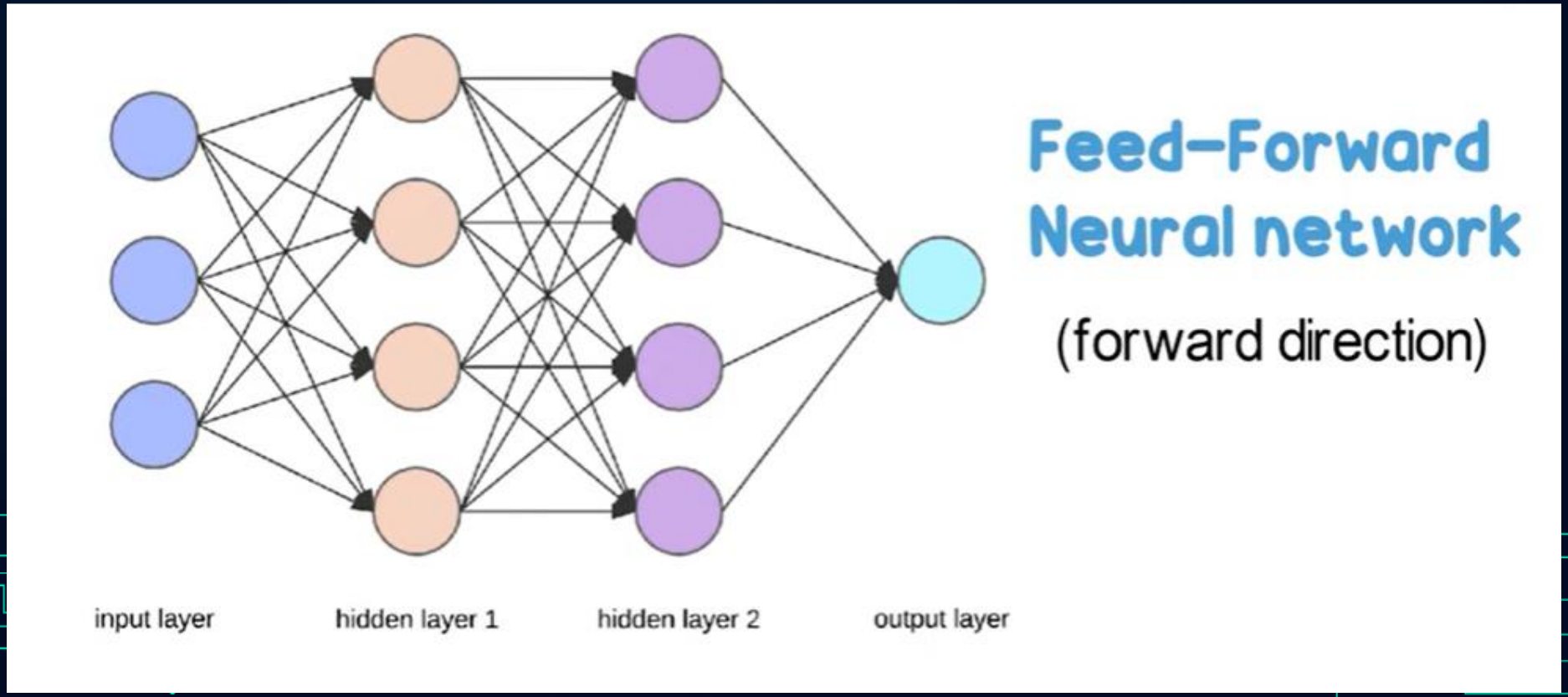
Types of DL

Major Types of Neural Network



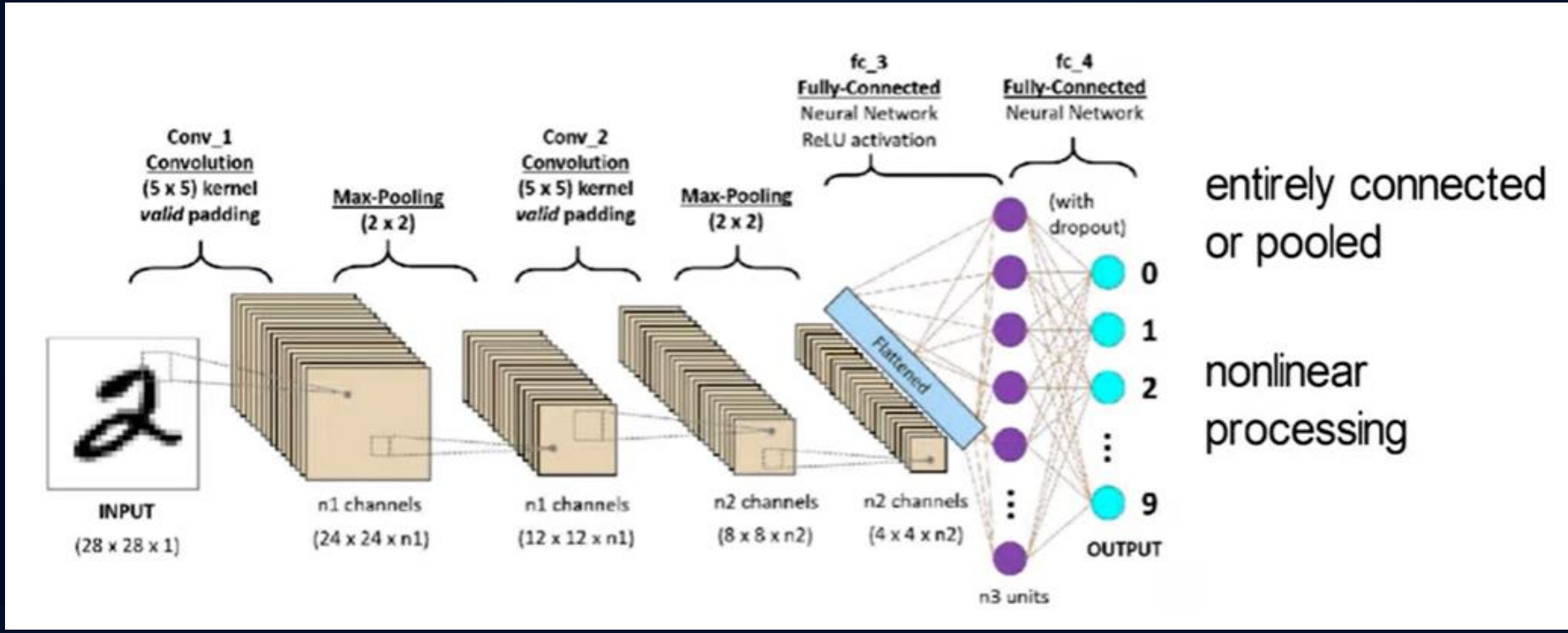
ANN (Artificial Neural Network)

- Simple NN (Multi Layer Perceptron)
- Useful for classification and Regression for Non-linear data



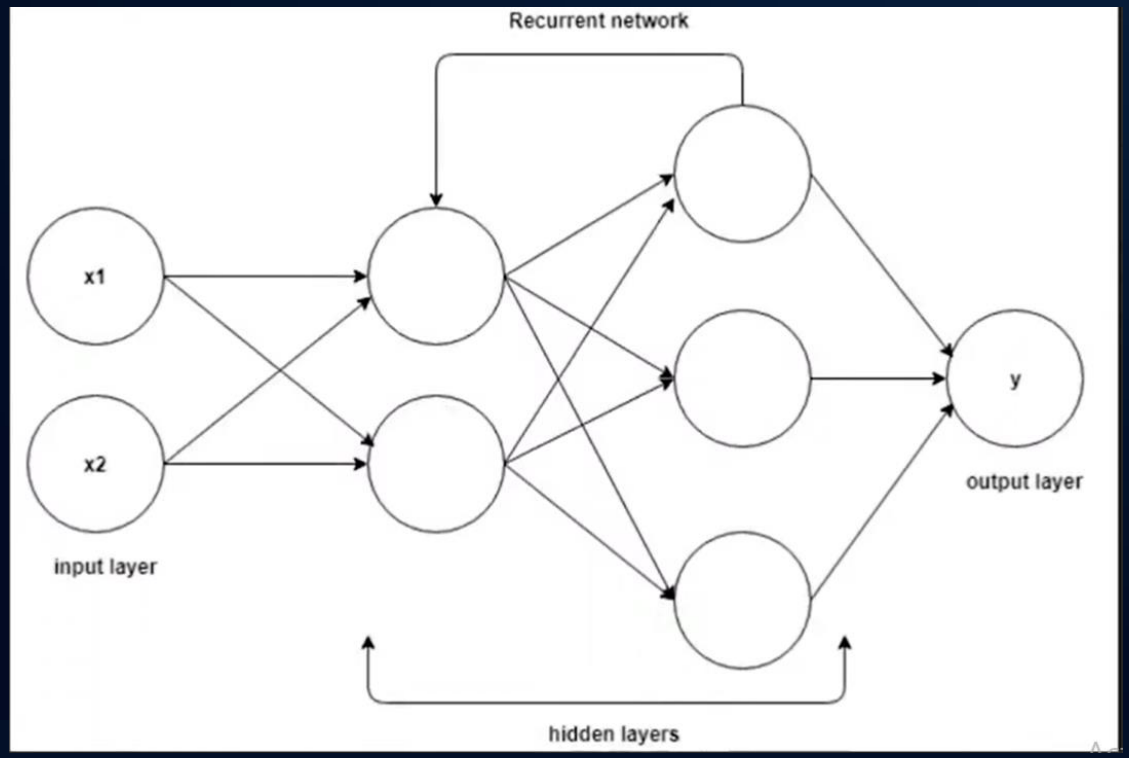
CNN (Convolutional Neural Network)

- Useful for image and video



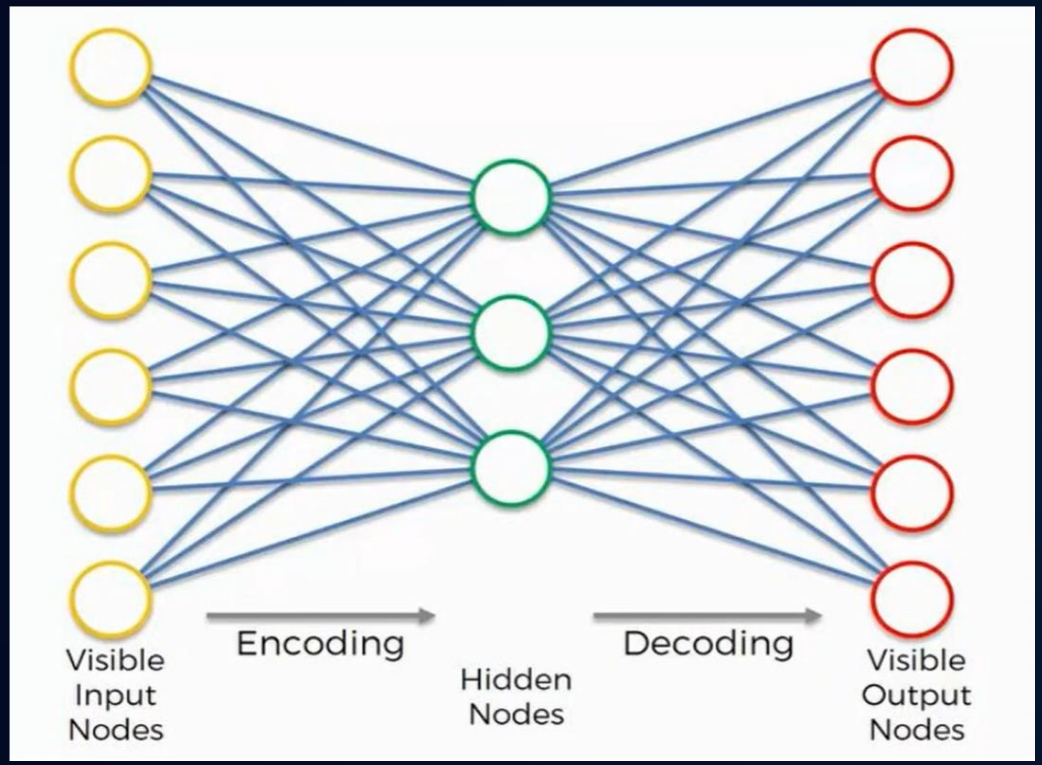
RNN (Recurrent Neural Network)

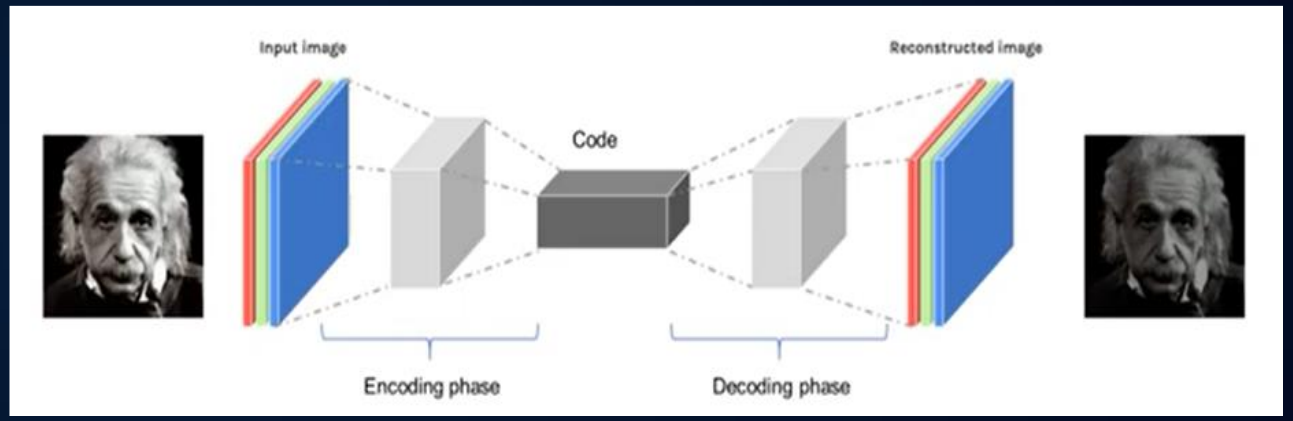
- Useful for NLP



Autoencoder

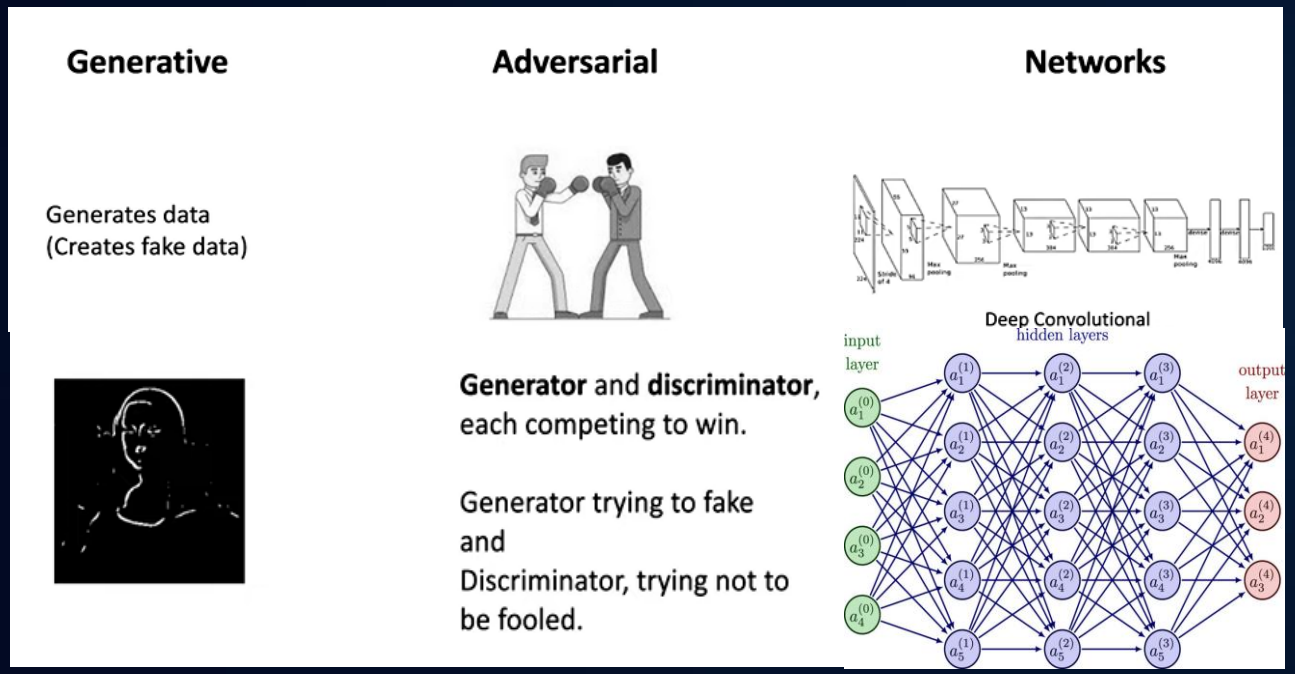
- Useful for Image and text compression

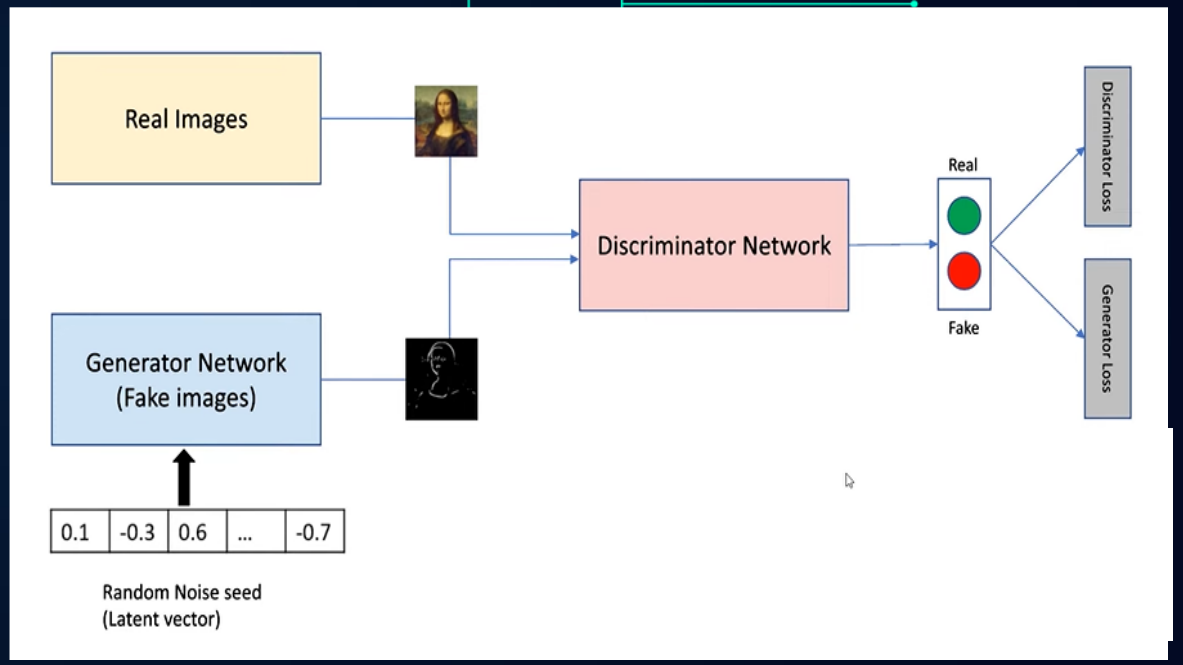




GAN (Generative Adversarial Network)

- Useful to generate fake image
- Text to image

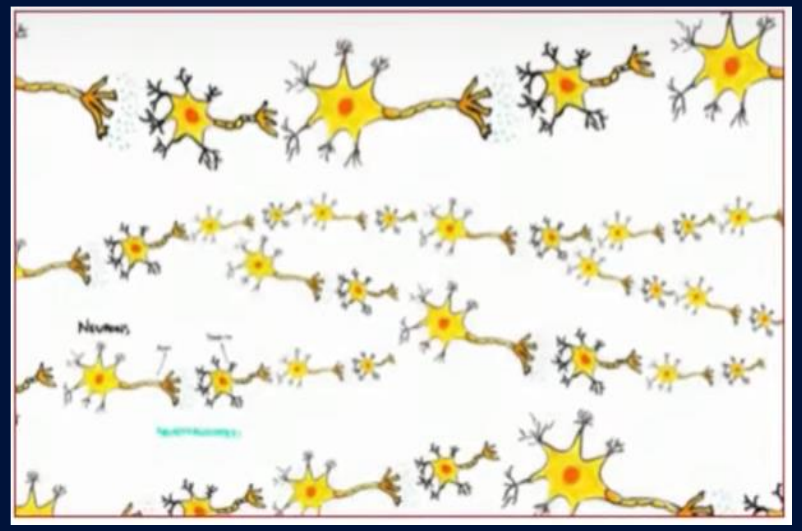


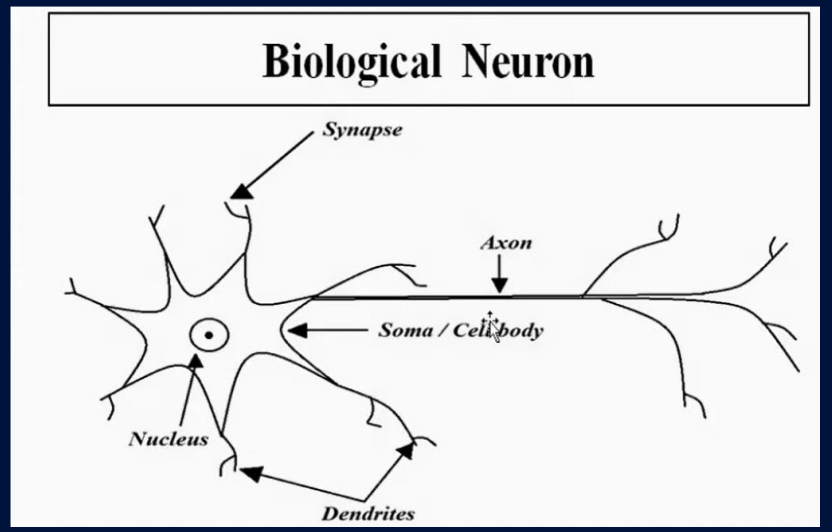


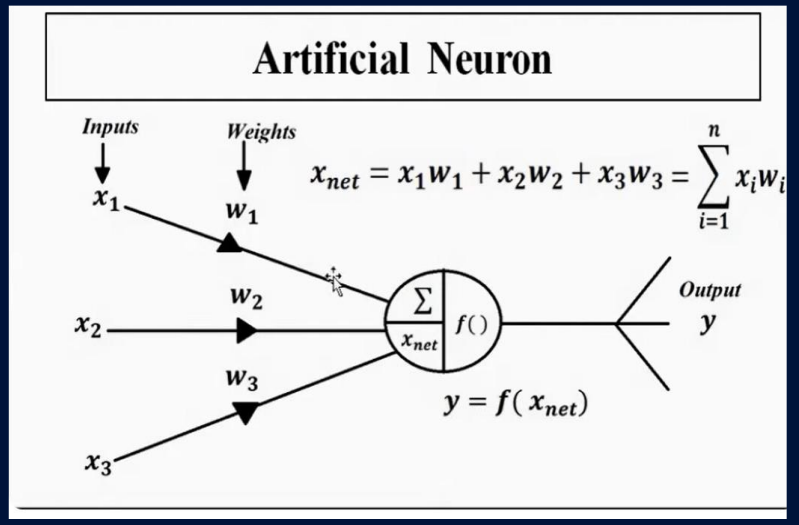
https://this-person-does-not-exist.com/en

https://deepai.org/machine-learning-model/text2imgd

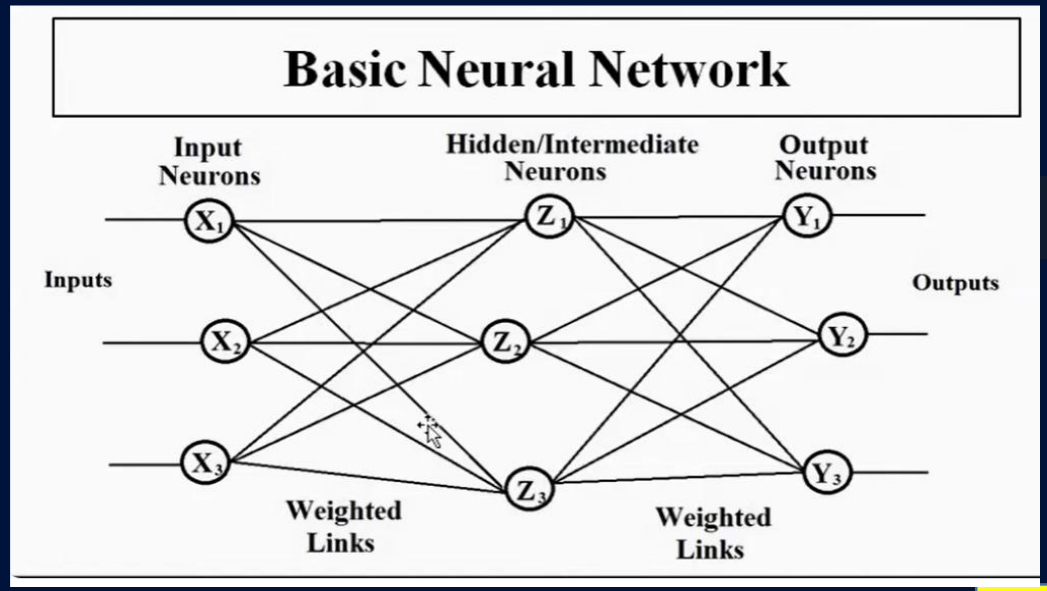
What is Neural Networks?







Biological Neuron	Artificial Neuron
Cell	Neuron / Node
Dendrites / Synapse	Weights
Soma	Net Input
Axon	Output



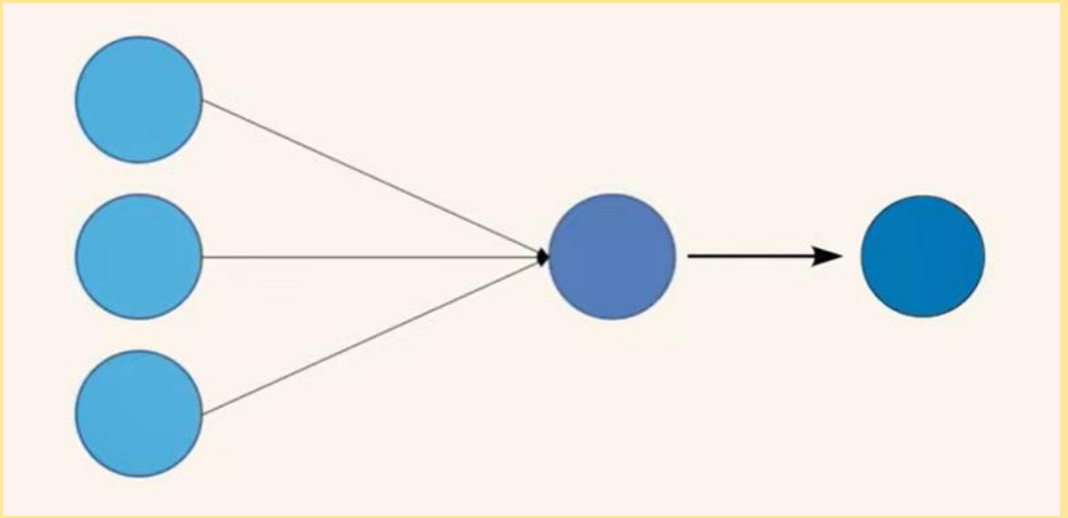
What is Neural Network?

Week-13, c-3

- **How to identify the following picture using NN?**



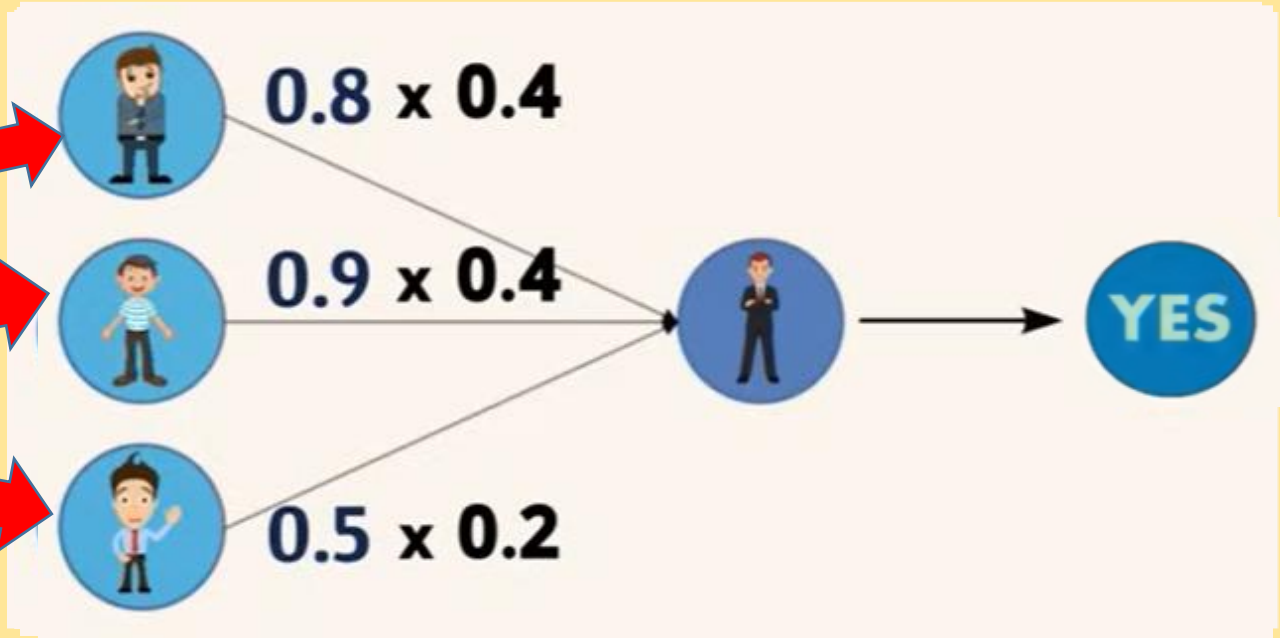




What is Neural Network?

Week-13, c-3





Face = (কান X 0.4) + (চোখ X 0.4) + (নাক X 0.2)

Face = (0.8 X 0.4) + (0.9 X 0.4) + (0.5 X 0.2)

Face = 0.78

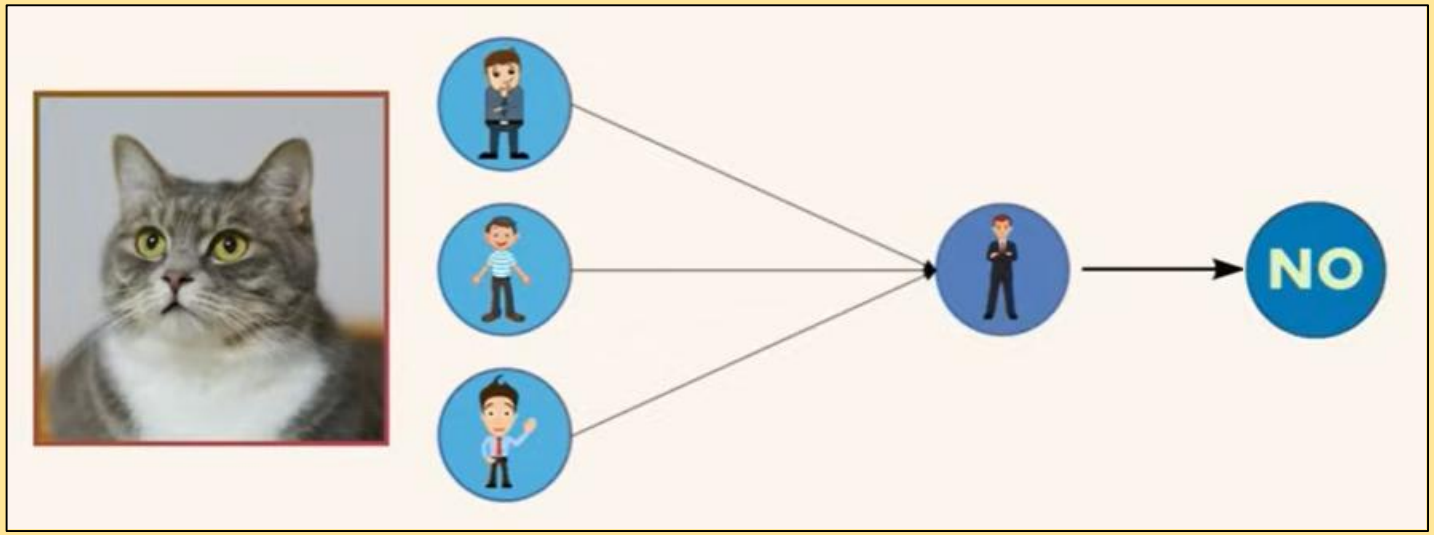
Output>0.5

So Panda

What is Neural Network?

Week-13, c-3

- **Identify the following picture**



Face = (কান X 0.4) + (চোখ X 0.4) + (নাক X 0.2)

Face = (0.3 X 0.4) + (0.2 X 0.4) + (0.6 X 0.2)

Face = **0.32**

Output<0.5

So NOT Panda

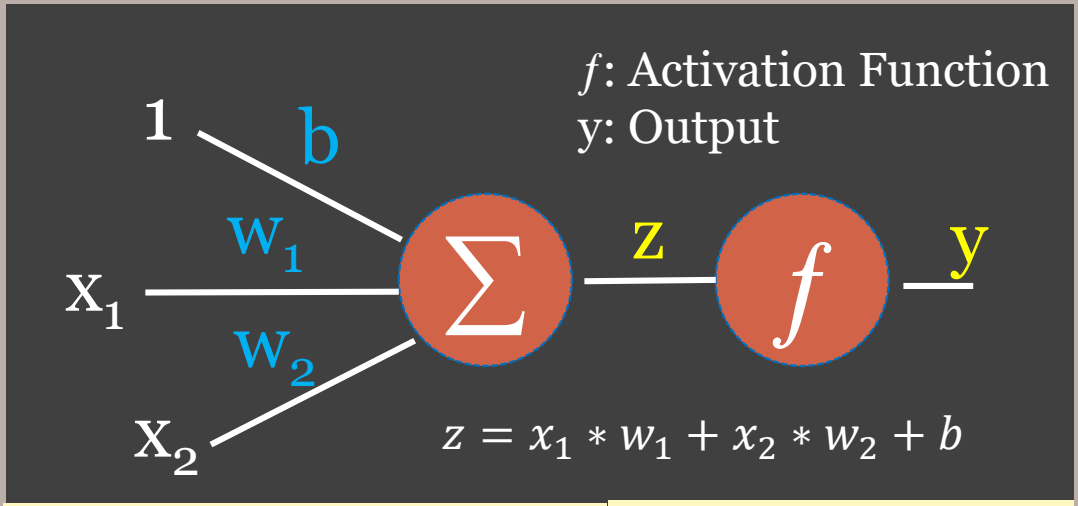


Deep Learning

Week-13, c-3

What is Perceptron?

Perceptron is a building block of an Artificial Neural Network.
Perceptron is also understood as a mathematical model of an Artificial Neuron or neural network unit .



Here
x_1,x2,1 are inputs
w_1,w_2, b are weights

For step function

$$y = \begin{cases} 0 & if\ z < 0 \\ 1 & if\ z \geq 0 \end{cases}$$

Key Definations

•**Input Nodes or Input Layer:**
This is the primary component of Perceptron which accepts the initial data into the system for further processing. Each input node contains a real numerical value.

•**Wight and Bias:**
Weight parameter represents the strength of the connection between units. This is another most important parameter of Perceptron components. Weight is directly proportional to the strength of the associated input neuron in deciding the output. Further, Bias can be considered as the line of intercept in a linear equation.

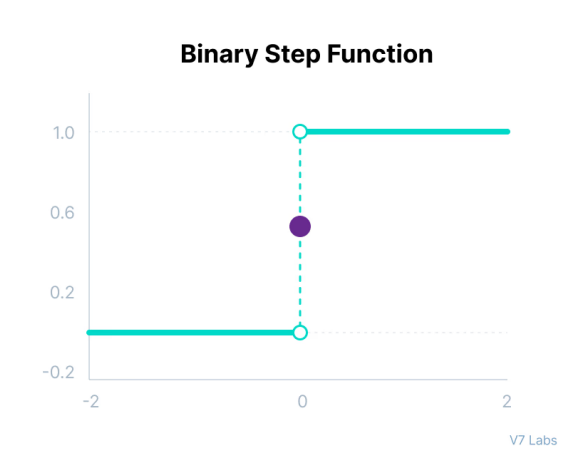
•**Activation Function:**
These are the final and important components that help to determine whether the neuron will fire or not. Activation Function can be considered primarily as a step function.

Deep Learning

Week-13, c-3

Some Activation Functions

1. **Binary Step Function**: The input fed to the activation function is compared to a certain threshold; if the input is greater than it, then the neuron is activated, else it is deactivated, meaning that its output is not passed on to the next hidden layer.



Binary step

$$f(x) = \begin{cases} 0 & for\ x < 0 \\ 1 & for\ x \geqslant 0 \end{cases}$$

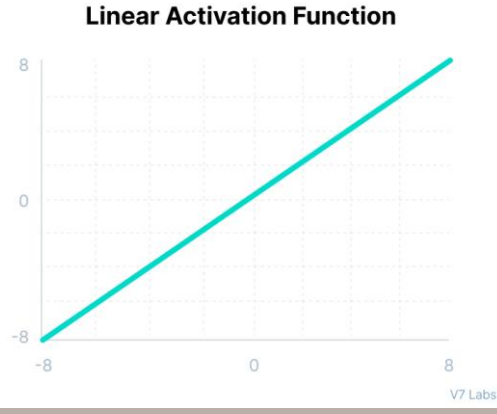
1. The input fed to the activation function is compared to a certain threshold; if the input is greater than it, then the neuron is activated, else it is deactivated, meaning that its output is not passed on to the next hidden layer.

Limitations

- It cannot provide multi-value outputs—for example, it cannot be used for multi-class classification problems.
- The gradient of the step function is zero, which causes a hindrance in the backpropagation process.

2. **Linear Activation Function**

The linear activation function, also known as "no activation," or "identity function" (multiplied x1.0), is where the activation is proportional to the input.
The function doesn't do anything to the weighted sum of the input, it simply spits out the value it was given.



Linear

$$f(x) = x$$

Deep Learning

Week-13, c-3

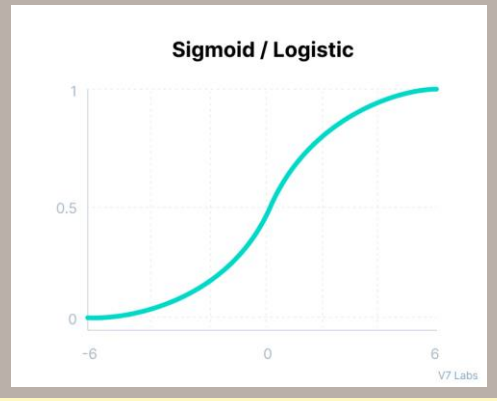
Limitations

- It's not possible to use backpropagation as the derivative of the function is a constant and has no relation to the input x.
- All layers of the neural network will collapse into one if a linear activation function is used. No matter the number of layers in the neural network, the last layer will still be a linear function of the first layer. So, essentially, a linear activation function turns the neural network into just one layer.

3 Sigmoid / Logistic Activation Function

This function takes any real value as input and outputs values in the range of 0 to 1.

The larger the input (more positive), the closer the output value will be to 1.0, whereas the smaller the input (more negative), the closer the output will be to 0.0.



Sigmoid / Logistic

$$f(x) = \frac{1}{1 + e^{-x}}$$

Advantage

- It is commonly used for models where we have to predict the probability as an output. Since probability of anything exists only between the range of 0 and 1, sigmoid is the right choice because of its range.

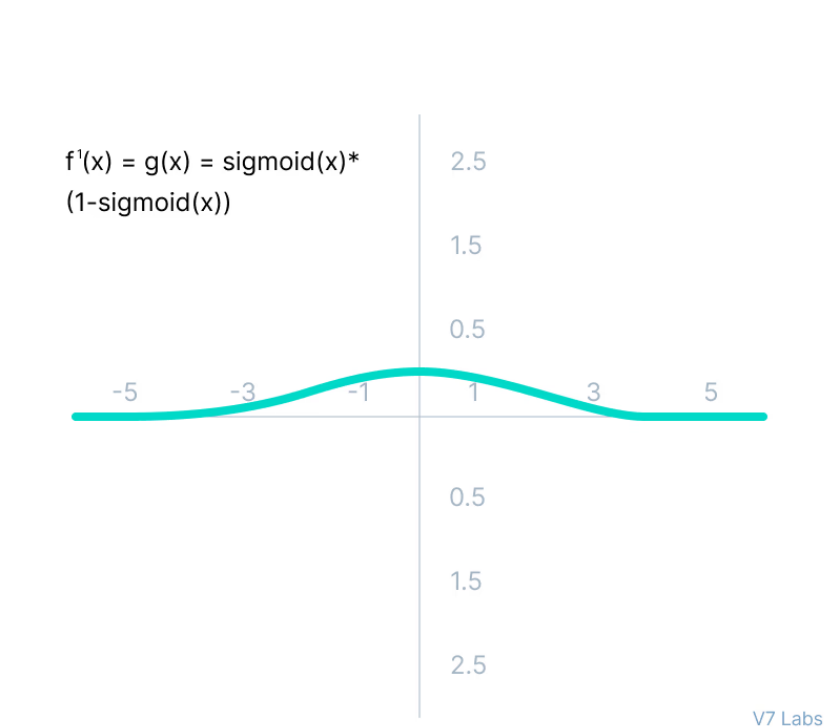
- The function is differentiable and provides a smooth gradient, i.e., preventing jumps in output values. This is represented by an S-shape of the sigmoid activation function.

Deep Learning

Week-13, c-3

Limitations

- The derivative of the function is f'(x) = sigmoid(x)*(1-sigmoid(x)).



As we can see from the above Figure, the gradient values are only significant for range -3 to 3, and the graph gets much flatter in other regions.

It implies that for values greater than 3 or less than -3, the function will have very small gradients. As the gradient value approaches zero, the network ceases to learn and suffers from the Vanishing gradient problem.

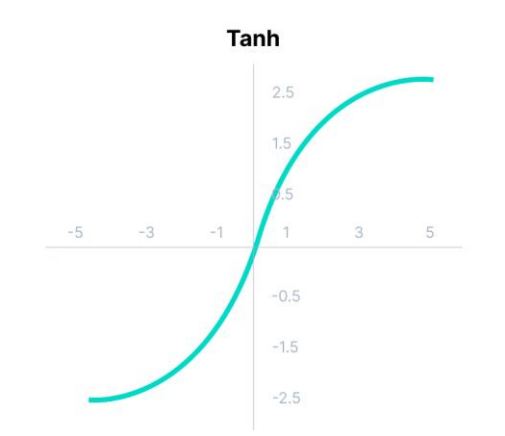
- The output of the logistic function is not symmetric around zero. So the output of all the neurons will be of the same sign. This makes the training of the neural network more difficult and unstable.

Deep Learning

Week-13, c-3

4 Tanh function

It is very similar to the sigmoid/logistic activation function, and even has the same S-shape with the difference in output range of -1 to 1. In Tanh, the larger the input (more positive), the closer the output value will be to 1.0, whereas the smaller the input (more negative), the closer the output will be to -1.0.



$$Tanh$$

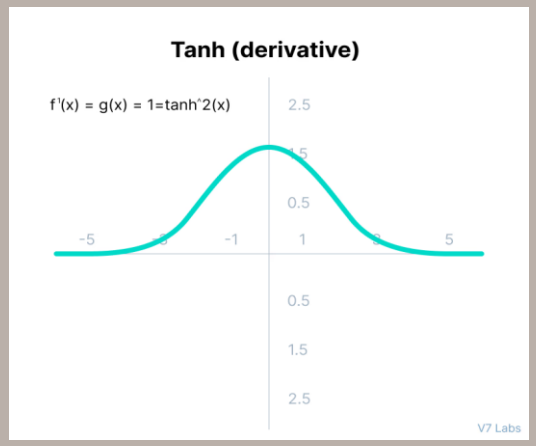
$$f(x) = \frac{(e^{x} - e^{-x})}{(e^{x} + e^{-x})}$$

Advantages

- The output of the tanh activation function is Zero centered; hence we can easily map the output values as strongly negative, neutral, or strongly positive.
- Usually used in hidden layers of a neural network as its values lie between -1 to; therefore, the mean for the hidden layer comes out to be 0 or very close to it. It helps in centering the data and makes learning for the next layer much easier.

Limitation

- Have a look at the gradient of the tanh activation function to understand its limitations.



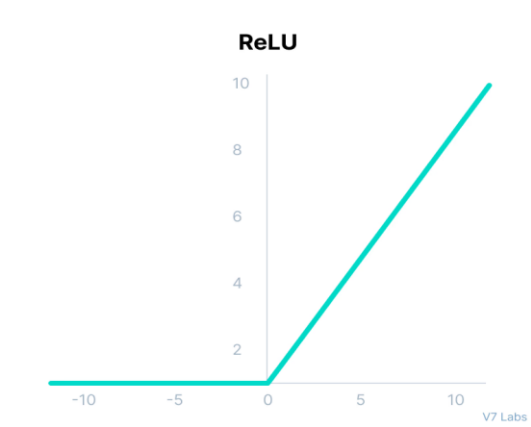
As you can see— it also faces the problem of vanishing gradients similar to the sigmoid activation function. Plus the gradient of the tanh function is much steeper as compared to the sigmoid function.

Deep Learning

Week-13, c-3

5 ReLU function

ReLU stands for Rectified Linear Unit.
Although it gives an impression of a linear function, ReLU has a derivative function and allows for backpropagation while simultaneously making it computationally efficient.
The main catch here is that the ReLU function does not activate all the neurons at the same time.
The neurons will only be deactivated if the output of the linear transformation is less than 0.



$$ReLU$$

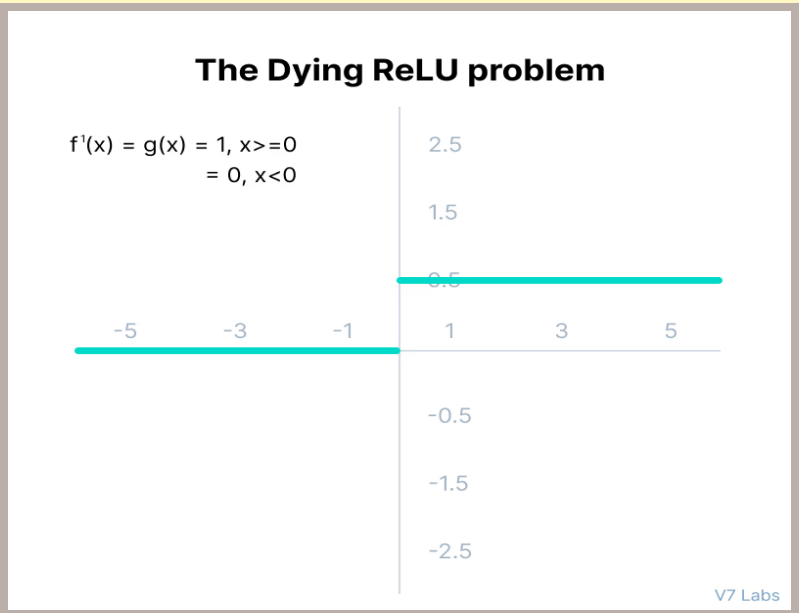
$$f(x) = max\,(0, x)$$

Advantages

- Since only a certain number of neurons are activated, the ReLU function is far more computationally efficient when compared to the sigmoid and tanh functions.
- ReLU accelerates the convergence of gradient descent towards the global minimum of the loss function due to its linear, non-saturating property.

Limitation

- The Dying ReLU problem, which I explained below.



Deep Learning

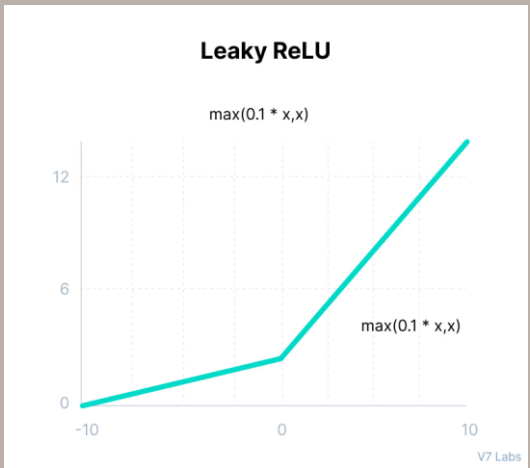
Week-13, c-3

The negative side of the graph makes the gradient value zero. Due to this reason, during the backpropagation process, the weights and biases for some neurons are not updated. This can create dead neurons which never get activated.

All the negative input values become zero immediately, which decreases the model's ability to fit or train from the data properly.

5.1 Leaky ReLU Function

Leaky ReLU is an improved version of ReLU function to solve the Dying ReLU problem as it has a small positive slope in the negative area.

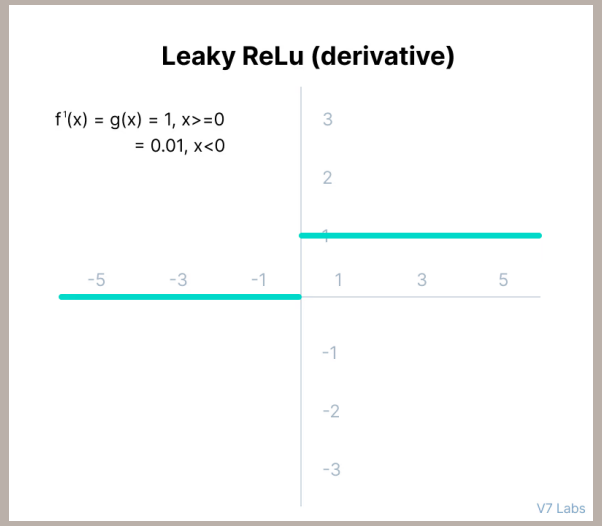


$$Leaky\ ReLU$$

$$f(x) = max\ (0.1x, x)$$

Advantages

- The advantages of Leaky ReLU are same as that of ReLU, in addition to the fact that it does enable backpropagation, even for negative input values.
- By making this minor modification for negative input values, the gradient of the left side of the graph comes out to be a non-zero value. Therefore, we would no longer encounter dead neurons in that region.
- Here is the derivative of the Leaky ReLU function.



Deep Learning

Week-13, c-3

Limitation

- The predictions may not be consistent for negative input values.
- The gradient for negative values is a small value that makes the learning of model parameters time-consuming.w.

Advantages

- The parameterized ReLU function is used when the leaky ReLU function still fails at solving the problem of dead neurons, and the relevant information is not successfully passed to the next layer.

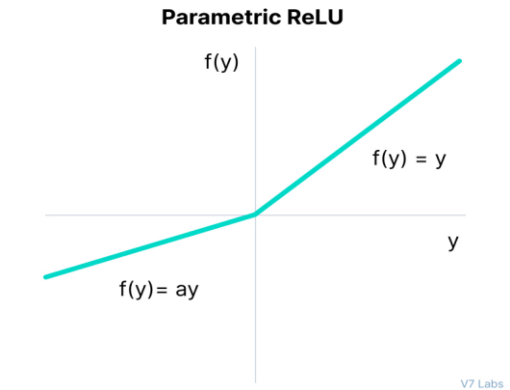
5.2 Parametric ReLU Function

Parametric ReLU is another variant of ReLU that aims to solve the problem of gradient's becoming zero for the left half of the axis.

This function provides the slope of the negative part of the function as an argument a. By performing backpropagation, the most appropriate value of a is learnt.

Limitation

- This function's limitation is that it may perform differently for different problems depending upon the value of slope parameter a.



Parametric ReLU

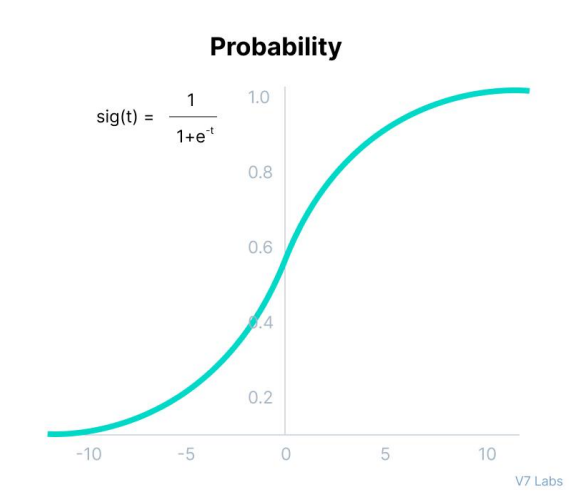
$$f(x) = max\,(ax, x)$$

Deep Learning

Week-13, c-3

6 Softmax Function

Before exploring the ins and outs of the Softmax activation function, we should focus on its building block—the sigmoid/logistic activation function that works on calculating probability values.



The output of the sigmoid function was in the range of 0 to 1, which can be thought of as probability.

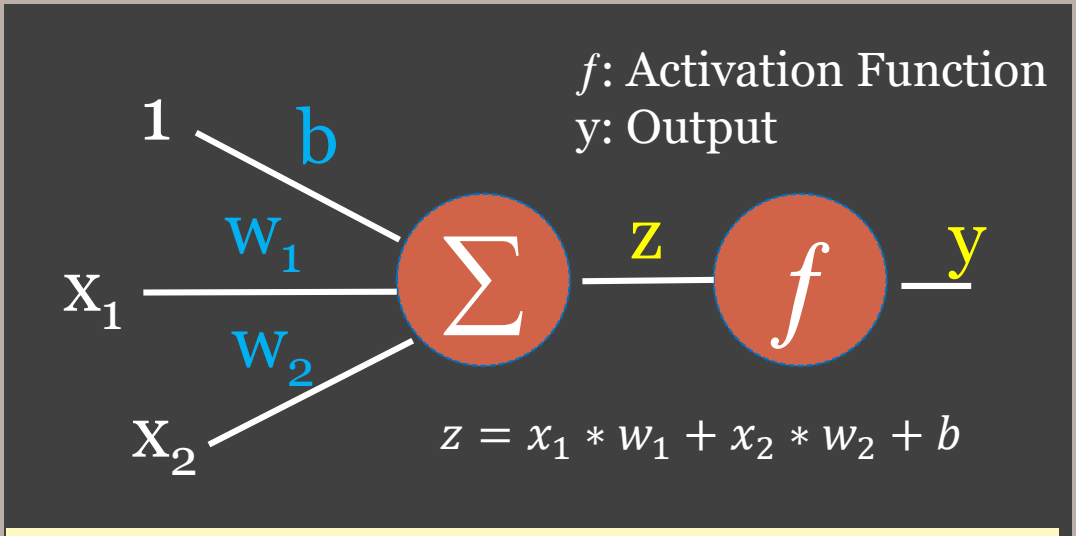
Deep Learning

Week-13, c-3

Example: Perceptron

Lets have a student placement dataset

IQ	CGPA	Placement
79	4.9	1
68	3.1	0



Let's consider first student
x_1=79, x2=4.9 and we have to find weights w_1,w_2 and bias b

Prediction of Model

- **Assume the values of weights and bias after training**

Lets w_1=1, w_2 =2 and bias b=3
Now predict the placement of a student whose IQ=100 and CGPA=3.5
→ *Calculate z=100*1+3.5*2+3=110*
Here z>0 so Placement=1

Deep Learning

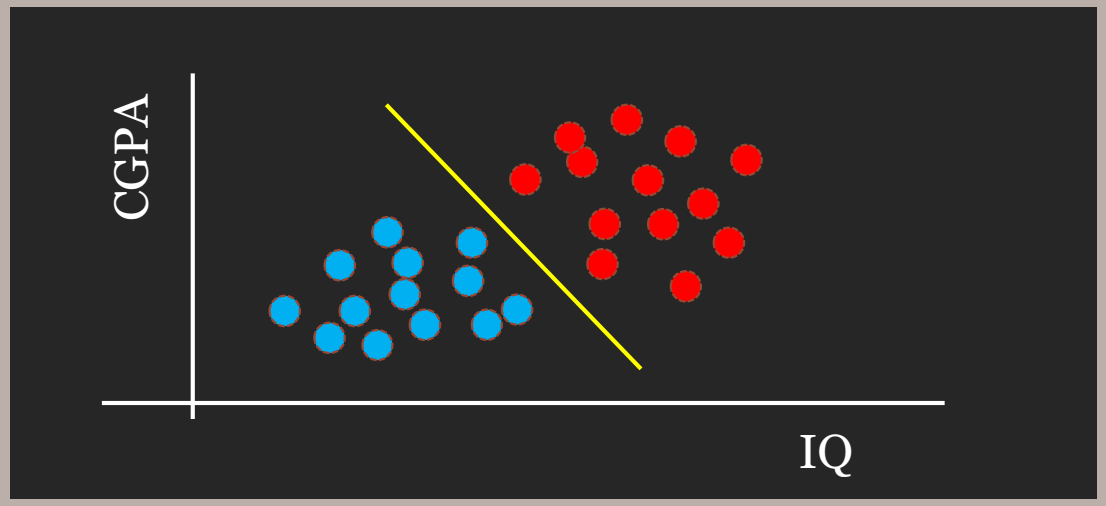
Week-13, c-3

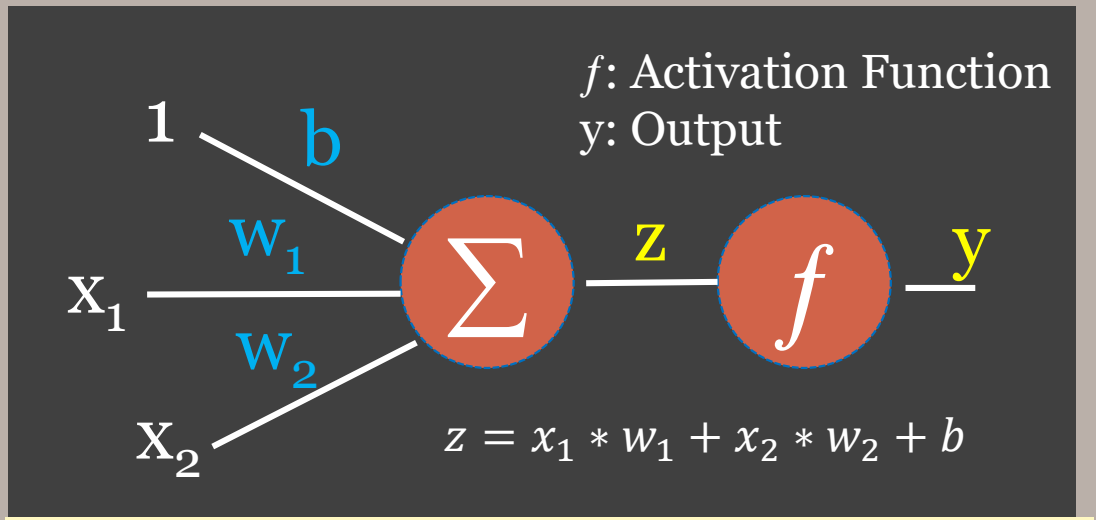
Geometric Intuition: Perceptron

Lets have a student placement dataset

IQ	CGPA	Placement
79	4.9	1
68	3.1	0

Now scatter plot the dataset where red and blue circle denotes Placement done and Placement not done respectively





Let's consider first student

$$y = f(z) = x_1 * w_1 + x_1 * w_1 + b = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$$

Let $x = x_1, A = w_1, y = x_2, B = w_2, C = b$
So, $z = Ax + Bx + C$, *which is a straight line*
Now if $Ax + By + C \geq 0$ *then placement* $= 1$
and if $Ax + By + C < 0$ *then placement=0*
The perceptron is a line that creates two regions, so the perceptron is a binary classifier

Deep Learning

Week-13, c-3

Example: Perceptron

If we have 3 features the equation becomes
$z = Ax + By + Cz + D$
So it is a plane. This place creates two regions where in one side placement=1 and in other side placement=0
If no. of features is more than 3 then it would be hyperplane.

Perceptron can separate linear data i.e. it a linear classifier but it can not classify non-linear dataset.

Deep Learning

Week-13, c-3

Python Example: Perceptron

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

```
df = pd.read_csv('f:\\python\\myplacement.csv')
```

```
print(df.shape)
df.head()
```

```
sns.scatterplot(df['cgpa'],df['resume_score'],hue=
df['placed'])
```

```
X = df.iloc[:,0:2]
y = df.iloc[:,-1]
```

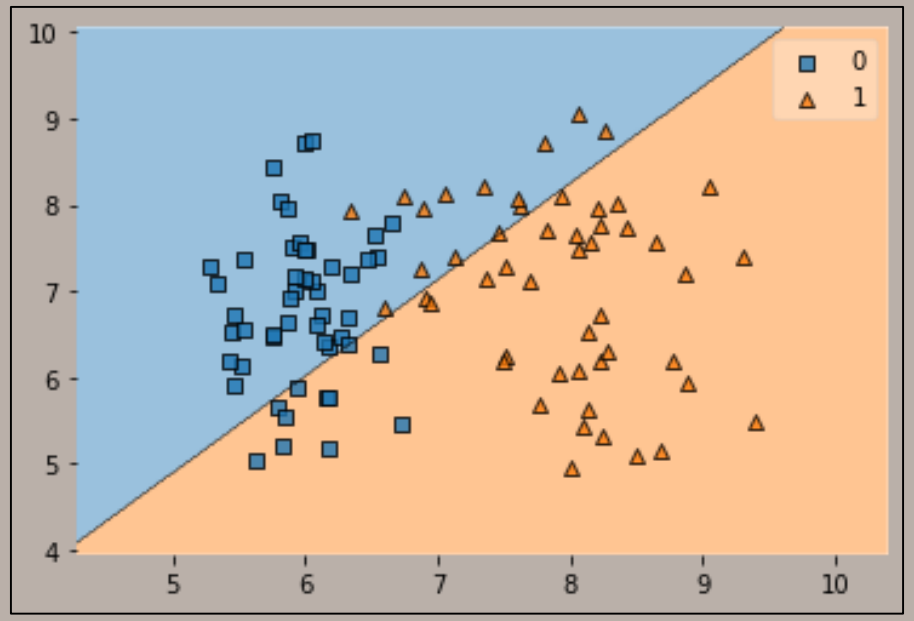
```
from sklearn.linear_model import Perceptron
p = Perceptron()
p.fit(X,y)
```

```
p.coef_
```

```
p.intercept_
```

```
from mlxtend.plotting import
plot_decision_regions
```

```
plot_decision_regions(X.values,        y.values,
clf=p, legend=1)
```



Week-13, c-3

End of Chapter